

**PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN MÔN HỌC

**ỨNG DỤNG KỸ THUẬT KHAI PHÁ DỮ LIỆU
TRONG NHẬN DẠNG BỆNH TRÊN LÁ CÂY
BẰNG MÔ HÌNH HỌC SÂU**

GVHD : Th. Vũ Thị Hạnh

LỚP : S2564CNTT

SVTH : PHAN VĂN VINH 2251068282

: DƯƠNG TRỌNG KHANH 2251068199

: TRẦN MINH TRÍ 2251068266

Hồ Chí Minh, Tháng 10 Năm 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC THỦY LỢI

ĐIỂM KẾT LUẬN		CHỮ KÝ XÁC NHẬN CỦA CÁN BỘ CHẤM THI SAU KHI CHẤM
(Ghi bằng số)	(Ghi bằng chữ)	
		Cán bộ chấm thi thứ 1: Cán bộ chấm thi thứ 2:

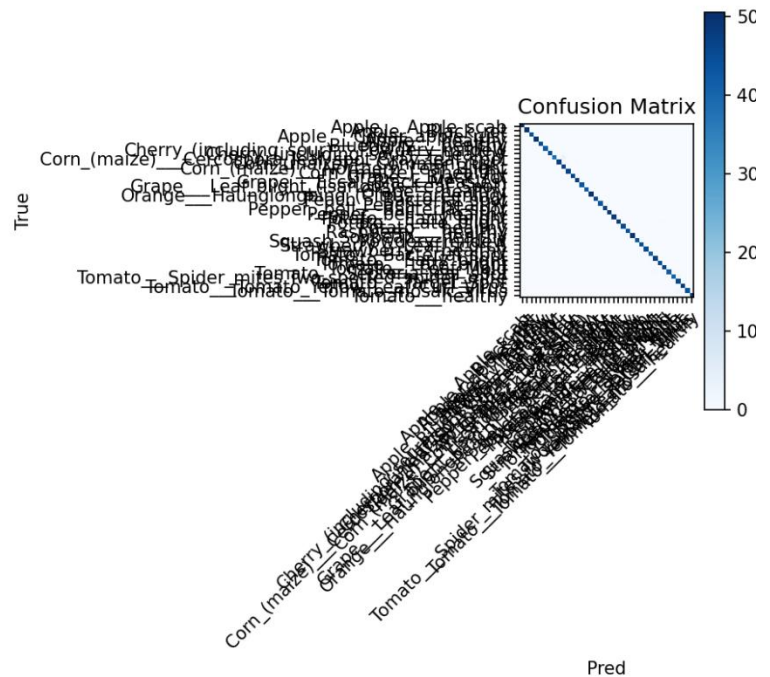
Chuyên ngành: Công nghệ thông tin
Báo cáo đồ án môn học: khai phá dữ liệu
Nhóm : BTS

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 10/2025

MỤC LỤC

CHƯƠNG I: CƠ SỞ LÝ THUYẾT.....	9
1.1. Khái niệm về khai phá dữ liệu (Data Mining).....	9
1.1.1. Định nghĩa khai phá dữ liệu.....	9
1.1.2. Các nhiệm vụ chính trong khai phá dữ liệu.....	9
1.2. Các bước trong quy trình khai phá dữ liệu	10
1.2.1. Thu thập dữ liệu (Data Collection).....	10
1.2.2. Tiền xử lý dữ liệu (Data Preprocessing)	10
1.2.3. Phát hiện tri thức (Knowledge Discovery)	11
1.3. Giới thiệu học sâu và mạng nơ-ron tích chập (CNN).....	11
1.3.1. Mạng nơ-ron nhân tạo (Artificial Neural Networks).....	11
1.3.2. Convolutional Neural Networks (CNN).....	12
1.3.3. Backpropagation và Gradient Descent	12
1.4. Tổng quan các mô hình Transfer Learning.....	13
1.4.1. Khái niệm Transfer Learning.....	13
1.4.2. Các kiến trúc CNN phổ biến.....	13
1.4.3. Vision Transformer (ViT)	14
1.4.4. Các phương pháp Transfer Learning	14
1.5. Một số nghiên cứu liên quan về phát hiện bệnh cây bằng Deep Learning.....	15
1.5.1. Các nghiên cứu tiên phong	15
1.5.2. Các nghiên cứu gần đây.....	15
1.5.3. Xu hướng nghiên cứu hiện tại	15
1.5.4. Thách thức và hướng phát triển	16

CHƯƠNG 2: DỮ LIỆU VÀ PHƯƠNG PHÁP NGHIÊN CỨU	17
2.1. Giới thiệu bộ dữ liệu "New Plant Diseases Dataset"	17
2.1.1. Nguồn dữ liệu	17
2.1.2. Thống kê dữ liệu	17
2.1.3. Cấu trúc thư mục.....	17
2.2. Tiền xử lý dữ liệu.....	18
2.2.1. Làm sạch và tổ chức dữ liệu	18
2.2.2. Chuẩn bị tập huấn luyện và kiểm định	18
2.2.3. Tăng cường dữ liệu (Data Augmentation)	19
2.2.4. Chuẩn bị đầu vào cho mô hình	19
2.3.1. Mô hình CNN tự xây dựng.....	20
2.3.2. Mô hình MobileNetV3-Small (Transfer Learning cho phân loại bệnh theo loài).....	23
2.3.3. Mô hình CAFormer-s18 (Hybrid CNN–Transformer Model).....	26
CHƯƠNG 3: THỰC NGHIỆM VÀ KẾT QUẢ.....	32
3.1. Môi trường thực nghiệm.....	32
3.1.1. Nền tảng thực nghiệm.....	32
3.1.2. Cấu hình phần cứng	32
3.1.3. Thư viện và phiên bản	32
3.2. Quá trình huấn luyện mô hình CNN.....	32
3.2.1. Kết quả thực nghiệm.....	33
3.2.2. Đánh giá mô hình.....	34
3.3. Quá trình huấn luyện mô hình MobileNetV3-Small	35
3.3.1. Kết quả thực nghiệm.....	35



3.3.2. Đánh giá mô hình	36
3.4. Quá trình huấn luyện mô hình CAFormer-s18	37
3.4.1. Kết quả thực nghiệm	37
3.4.2. Đánh giá mô hình	38
3.5. So sánh tổng hợp giữa các mô hình	39
3.5.1. Phân tích và nhận xét	40
3.5.2. Tổng kết	41
CHƯƠNG 4: DEPLOY WEB	42
4.1. Tổng quan về hệ thống triển khai	42
4.1.2. Công nghệ sử dụng	42
4.2. Thiết kế giao diện người dùng	42
4.2.1. Cấu trúc giao diện	42
4.2.2. Tính năng tương tác	43
4.3. Triển khai mô hình	45

4.3.1. Hỗ trợ đa mô hình.....	45
4.3.2. Xử lý ảnh đầu vào.....	45
4.3.3. Chuyển đổi mô hình động.....	46
4.4. Tính năng Grad-CAM.....	47
4.4.1. Giải thích dự đoán.....	47
4.4.2. Visualization	48
4.5. Triển khai và cài đặt.....	48
4.5.1. Yêu cầu hệ thống	48
4.5.2. Cài đặt dependencies	49
4.5.3. Cấu trúc thư mục.....	50
4.6. Hướng dẫn sử dụng.....	50
4.6.1. Khởi động ứng dụng	50
4.6.2. Quy trình sử dụng	51
4.7. Tối ưu hóa và mở rộng.....	51
4.7.1. Tối ưu hóa hiệu suất.....	52
4.8. Đánh giá và kết quả	52
4.8.1. Hiệu suất triển khai.....	52
4.8.2. Trải nghiệm người dùng	53
CHƯƠNG: 5 KẾT LUẬN	54
5.1. Tổng kết kết quả nghiên cứu.....	54
5.2. Đánh giá mô hình và hạn chế	54
5.3. Hướng phát triển và mở rộng.....	55

Danh mục các bảng

hình 1 cấu trúc thư mục data.....	18
hình 2: biểu đồ logic chuyển giai đoạn.....	29
hình 3: các checkpoint được tải về	30
hình 4: Sơ đồ quy trình huấn luyện.....	30
hình 5:Ma trận nhầm lẫn của mô hình CNN.....	34
hình 6: Ma trận nhầm lẫn của mô hình MobileNetV3-Small	36
hình 7: ma trận nhầm lẫn CAFormer	38
hình 8: tính năng tương tác	43
hình 9: up load ảnh đa dạng	44
hình 10: kết quả dự đoán và grad cam	44
hình 11: cấu trúc thư mục dự án	50

LỜI NÓI ĐẦU

Trong bối cảnh nông nghiệp hiện đại, việc phát hiện sớm và chính xác các bệnh trên cây trồng đóng vai trò quan trọng trong việc đảm bảo năng suất và chất lượng nông sản. Tuy nhiên, việc nhận diện bệnh lá cây dựa trên quan sát trực quan thường gặp nhiều khó khăn do sự tương đồng giữa các triệu chứng, sự thay đổi điều kiện môi trường và yêu cầu cao về chuyên môn của người thực hiện.

Sự phát triển mạnh mẽ của trí tuệ nhân tạo (AI) và học máy (Machine Learning), đặc biệt là học sâu (Deep Learning) đã mở ra hướng tiếp cận mới trong việc tự động hóa quá trình chẩn đoán bệnh cây trồng. Các mô hình học sâu có khả năng trích xuất đặc trưng phức tạp từ dữ liệu hình ảnh, giúp cải thiện đáng kể độ chính xác và khả năng tổng quát hóa của hệ thống phân loại bệnh.

Đề án này tập trung nghiên cứu và triển khai các mô hình học sâu nhằm phân loại 38 loại bệnh lá cây dựa trên bộ dữ liệu New Plant Diseases Dataset từ Kaggle. Trong đó, mô hình CNN tự xây dựng được sử dụng làm đường cơ sở để so sánh, MobilenetV3_small đại diện cho hướng tối ưu hóa mô hình gọn nhẹ, và CAFormer-s18 là mô hình tiên tiến kết hợp giữa CNN và Transformer, có khả năng học đồng thời đặc trưng không gian và ngữ cảnh sâu trong hình ảnh.

Quá trình nghiên cứu bao gồm các bước đặc trưng của khai phá dữ liệu: tiền xử lý và tăng cường dữ liệu, xây dựng mô hình, huấn luyện – đánh giá hiệu năng, và trực quan hóa kết quả bằng các kỹ thuật giải thích mô hình như Grad-CAM. Kết quả cho thấy CAFormer-s18 đạt độ chính xác 99.33% trên tập validation, vượt trội so với CNN tự xây và MobilenetV3_small, khẳng định tính hiệu quả của kiến trúc kết hợp giữa tích chập và cơ chế chú ý trong bài toán nhận dạng bệnh lá cây.

Mục tiêu của đề án không chỉ hướng đến việc xây dựng mô hình phân loại chính xác, mà còn nhằm tạo nền tảng cho hệ thống hỗ trợ chẩn đoán bệnh cây trồng tự động, góp phần ứng dụng trí tuệ nhân tạo vào nông nghiệp thông minh và nâng cao hiệu quả sản xuất bền vững.

CHƯƠNG I: CƠ SỞ LÝ THUYẾT

1.1. Khái niệm về khai phá dữ liệu (Data Mining)

1.1.1. Định nghĩa khai phá dữ liệu

Khai phá dữ liệu (Data Mining) là quá trình khám phá và trích xuất tri thức có ý nghĩa từ các tập dữ liệu lớn thông qua việc sử dụng các kỹ thuật thống kê, học máy và trí tuệ nhân tạo. Theo định nghĩa của Fayyad và cộng sự (1996), khai phá dữ liệu là "quá trình khám phá các mẫu có ý nghĩa và tri thức từ các tập dữ liệu lớn".

1.1.2. Các nhiệm vụ chính trong khai phá dữ liệu

Phân loại (Classification):

- Mục tiêu: Dự đoán class/category của một đối tượng dựa trên các thuộc tính đã biết
- Ví dụ: Phân loại bệnh lá cây dựa trên hình ảnh
- Các thuật toán: Decision Tree, Random Forest, SVM, Neural Networks

Phân cụm (Clustering):

- Mục tiêu: Nhóm các đối tượng tương tự nhau vào cùng một cluster
- Ví dụ: Nhóm các loại bệnh có triệu chứng tương tự
- Các thuật toán: K-means, Hierarchical Clustering, DBSCAN

Dự đoán (Prediction):

- Mục tiêu: Dự đoán giá trị số liên tục
- Ví dụ: Dự đoán mức độ nghiêm trọng của bệnh
- Các thuật toán: Linear Regression, Polynomial Regression, Neural Networks

Phát hiện quy luật (Association Rules):

- Mục tiêu: Tìm ra các mối quan hệ giữa các thuộc tính
- Ví dụ: Các triệu chứng thường xuất hiện cùng nhau

- Các thuật toán: Apriori, FP-Growth

1.2. Các bước trong quy trình khai phá dữ liệu

1.2.1. Thu thập dữ liệu (Data Collection)

Nguồn dữ liệu:

- Dữ liệu cấu trúc: Cơ sở dữ liệu quan hệ, CSV files
- Dữ liệu phi cấu trúc: Hình ảnh, văn bản, âm thanh
- Dữ liệu bán cấu trúc: JSON, XML

1.2.2. Tiền xử lý dữ liệu (Data Preprocessing)

Data Cleaning:

- Loại bỏ ảnh bị lỗi hoặc chất lượng kém
- Kiểm tra và sửa lỗi labeling
- Xử lý ảnh bị nhiễu hoặc mờ

Normalization:

- Chuẩn hóa kích thước ảnh về cùng một kích thước
- Normalize pixel values về khoảng $[0,1]$
- Áp dụng ImageNet normalization cho transfer learning

Data Augmentation:

- Geometric Transformations:
 - Random Resized Crop: Cắt và resize ngẫu nhiên
 - Random Horizontal Flip: Lật ngang với xác suất 50%
 - Rotation: Xoay hình ảnh
- Color Transformations:
 - ColorJitter: Thay đổi brightness, contrast, saturation, hue

- Random Erasing: Xóa ngẫu nhiên một vùng trong ảnh
- Advanced Augmentation:
 - RandAugment: Tự động chọn và áp dụng các phép biến đổi
 - Mixup: Trộn hai ảnh và label tương ứng
 - CutMix: Cắt và dán một vùng từ ảnh này sang ảnh khác

1.2.3. Phát hiện tri thức (Knowledge Discovery)

Modeling:

- Lựa chọn thuật toán phù hợp
- Thiết kế kiến trúc mô hình
- Huấn luyện mô hình với dữ liệu đã xử lý

Evaluation:

- Đánh giá hiệu suất mô hình
- Sử dụng các metrics: Accuracy, Precision, Recall, F1-Score
- Phân tích confusion matrix
- Cross-validation để đảm bảo tính ổn định

1.3. Giới thiệu học sâu và mạng nơ-ron tích chập (CNN)

1.3.1. Mạng nơ-ron nhân tạo (Artificial Neural Networks)

Cấu trúc cơ bản:

- Input Layer: Nhận dữ liệu đầu vào
- Hidden Layers: Các lớp ẩn xử lý thông tin
- Output Layer: Đưa ra kết quả dự đoán

Hàm kích hoạt (Activation Functions):

- ReLU: $f(x) = \max(0, x)$ - phổ biến nhất

- Sigmoid: $f(x) = 1/(1 + e^{(-x)})$ - cho binary classification
- Softmax: $f(x_i) = e^{(x_i)} / \sum e^{(x_j)}$ - cho multi-class classification

1.3.2. Convolutional Neural Networks (CNN)

Các thành phần chính:

- Convolutional Layer:
 - Sử dụng các kernel (filter) để trích xuất đặc trưng
 - Công thức: $(I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) * K(m,n)$
 - Tạo ra feature maps chứa thông tin về các pattern cục bộ
- Pooling Layer:
 - Max Pooling: Lấy giá trị lớn nhất trong vùng
 - Average Pooling: Lấy giá trị trung bình trong vùng
 - Mục đích: Giảm kích thước, tăng tính bất biến
- Fully Connected Layer:
 - Kết nối tất cả neurons từ layer trước
 - Thực hiện phân loại cuối cùng
 - Số lượng neurons = số classes cần phân loại

1.3.3. Backpropagation và Gradient Descent

Backpropagation:

- Thuật toán lan truyền ngược để tính gradient
- Cập nhật trọng số dựa trên gradient của loss function

Gradient Descent:

- Batch GD: Cập nhật sau khi xử lý toàn bộ dataset
- Stochastic GD: Cập nhật sau mỗi sample
- Mini-batch GD: Cập nhật sau mỗi batch nhỏ

1.4. Tổng quan các mô hình Transfer Learning

1.4.1. Khái niệm Transfer Learning

Transfer Learning là kỹ thuật sử dụng kiến thức đã học được từ một tác vụ (source task) để giải quyết một tác vụ khác có liên quan (target task). Trong computer vision, thường sử dụng các mô hình đã được pre-train trên ImageNet.

1.4.2. Các kiến trúc CNN phổ biến

Conformer S18 (Convolution + Transformer Hybrid Network):

- Kết hợp giữa Convolution để học local pattern và Self-Attention để học global context
- Sử dụng khối Macaron Feed-Forward + Multi-Head Attention + Convolution Module
- Ổn định gradient tốt nhờ cơ chế residual + normalization
- Thường dùng trong ASR (Automatic Speech Recognition) nhưng được mở rộng sang Vision

MobileNet:

- Được thiết kế cho mobile và embedded devices
- Sử dụng Depthwise Separable Convolution
- Giảm đáng kể số parameters và FLOPs
- MobileNetV1, MobilenetV3_small, MobileNetV3

EfficientNet:

- Tối ưu hóa cả accuracy và efficiency
- Sử dụng compound scaling method
- Các phiên bản từ EfficientNet-B0 đến EfficientNet-B7

DenseNet:

- Mỗi layer kết nối với tất cả các layer trước đó
- Tăng cường feature reuse
- Giảm số parameters

1.4.3. Vision Transformer (ViT)

Kiến trúc ViT:

- Chia ảnh thành các patches
- Thêm positional encoding
- Sử dụng Multi-Head Self-Attention
- Có thể đạt hiệu suất cao hơn CNN trên một số tác vụ

Convolutional Attention Transformer (CAFormer):

- Kết hợp ưu điểm của CNN và Transformer
- Sử dụng Convolutional Stem thay vì Linear Projection
- Hiệu quả hơn ViT thuần túy

1.4.4. Các phương pháp Transfer Learning

Feature Extraction:

- Freeze các layer đầu của pre-trained model
- Chỉ train classifier mới
- Phù hợp khi dataset nhỏ

Fine-tuning:

- Unfreeze một phần hoặc toàn bộ model
- Sử dụng learning rate thấp hơn
- Phù hợp khi có đủ dữ liệu

Progressive Unfreezing:

- Từ từ unfreeze các layer từ cuối lên đầu

- Cân bằng giữa stability và flexibility

1.5. Một số nghiên cứu liên quan về phát hiện bệnh cây bằng Deep Learning

1.5.1. Các nghiên cứu tiên phong

Mohanty et al. (2016):

- Sử dụng CNN để phân loại bệnh cây
- Dataset: PlantVillage
- Sử dụng GoogLeNet architecture

Sladojevic et al. (2016):

- Phát triển hệ thống phát hiện bệnh cây trồng
- Sử dụng Caffè framework
- Tập trung vào các loại cây quan trọng về kinh tế

1.5.2. Các nghiên cứu gần đây

Ferentinos (2018):

- So sánh hiệu suất của các kiến trúc CNN khác nhau
- Sử dụng PlantVillage dataset
- VGG, ResNet, AlexNet, GoogLeNet

Zhang et al. (2019):

- Áp dụng Transfer Learning với ResNet
- Sử dụng ImageNet pre-trained weights
- Đạt hiệu suất cao trên PlantVillage dataset

1.5.3. Xu hướng nghiên cứu hiện tại

Vision Transformer trong Plant Disease Detection:

- ViT và các biến thể (DeiT, Swin Transformer)

- Hiệu suất cạnh tranh với CNN
- Cần dataset lớn hơn để train từ đầu

Hybrid Architectures:

- Kết hợp CNN và Transformer
- CAFormer, ConViT, CvT
- Tận dụng ưu điểm của cả hai kiến trúc

Advanced Training Strategies:

- Multi-stage training
- Progressive unfreezing
- Learning rate scheduling
- Data augmentation nâng cao

1.5.4. Thách thức và hướng phát triển

Thách thức:

- Dataset không cân bằng giữa các classes
- Ảnh chụp trong điều kiện thực tế khác với lab
- Sự tương đồng giữa các loại bệnh khác nhau
- Yêu cầu về computational resources

CHƯƠNG 2: DỮ LIỆU VÀ PHƯƠNG PHÁP NGHIÊN CỨU

2.1. Giới thiệu bộ dữ liệu "New Plant Diseases Dataset"

2.1.1. Nguồn dữ liệu

Bộ dữ liệu được sử dụng trong nghiên cứu này là New Plant Diseases Dataset, một bộ dữ liệu công khai được lấy từ nguồn Kaggle. Đây là một bộ dữ liệu được sử dụng rộng rãi trong lĩnh vực phân loại bệnh cây trồng và đã được tăng cường (augmented) để cải thiện hiệu suất của các mô hình deep learning.

2.1.2. Thống kê dữ liệu

Tổng số ảnh: 87.867 ảnh.

- Training set: 70.295 ảnh (80%).
- Validation set: 17.572 ảnh (20%).

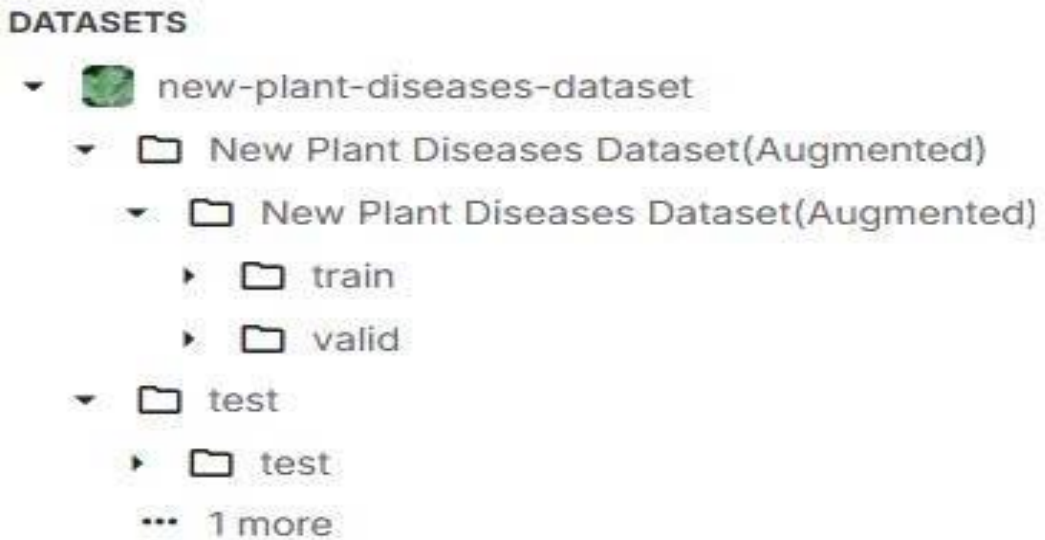
Số lớp (classes):

- 38 loại bệnh lá cây khác nhau, bao gồm các cây như Apple, Corn, Tomato, Grape, Potato, v.v...
- Mỗi loại cây có cả trạng thái "Healthy" (khỏe mạnh) và các loại bệnh đặc trưng khác.

Kích thước ảnh:

- Định dạng gốc: RGB images với kích thước đa dạng
- Chất lượng: Ảnh chất lượng cao, được chụp trong điều kiện lab

2.1.3. Cấu trúc thư mục



hình 1 cấu trúc thư mục data

2.2. Tiền xử lý dữ liệu

Quá trình tiền xử lý dữ liệu đóng vai trò quan trọng trong việc đảm bảo chất lượng đầu vào và nâng cao hiệu suất huấn luyện của các mô hình học sâu. Dữ liệu sau khi được thu thập từ bộ “New Plant Diseases Dataset” sẽ được xử lý qua các bước cơ bản sau:

2.2.1. Làm sạch và tổ chức dữ liệu

Trước hết, các ảnh bị lỗi, mờ hoặc có nhãn sai được loại bỏ để đảm bảo tính chính xác của bộ dữ liệu. Sau đó, dữ liệu được sắp xếp lại theo cấu trúc thư mục tương ứng với từng loại bệnh, giúp dễ dàng quản lý và nạp vào trong quá trình huấn luyện.

2.2.2. Chuẩn bị tập huấn luyện và kiểm định

Toàn bộ dữ liệu được chia thành hai phần:

- Tập huấn luyện (Training set): chiếm khoảng 80% tổng số ảnh.
- Tập kiểm định (Validation set): chiếm khoảng 20% tổng số ảnh.

Tỷ lệ này được lựa chọn để đảm bảo đủ dữ liệu cho quá trình học và đánh giá mô hình một cách công bằng.

2.2.3. Tăng cường dữ liệu (Data Augmentation)

Nhằm tăng tính đa dạng và giảm hiện tượng overfitting, quá trình tăng cường dữ liệu được áp dụng bằng cách sử dụng các phép biến đổi hình học và màu sắc ngẫu nhiên (ví dụ như xoay, lật, thay đổi độ sáng, độ tương phản, hoặc cắt xén). Việc tăng cường dữ liệu giúp mô hình học được đặc trưng ổn định và khái quát tốt hơn trong điều kiện ảnh thực tế.

2.2.4. Chuẩn bị đầu vào cho mô hình

Sau khi hoàn tất các bước trên, dữ liệu sẽ được chuyển sang giai đoạn xử lý đầu vào cụ thể cho từng mô hình học sâu. Mỗi kiến trúc (CNN tự xây dựng, MobilenetV3_small, CAFormer-s18) có yêu cầu riêng về kích thước ảnh, phương pháp chuẩn hóa và định dạng dữ liệu..

2.3. Mô hình sử dụng

Mô hình 1: CNN tự xây dựng

- Kiến trúc CNN truyền thống
- Được thiết kế từ đầu cho tác vụ phân loại bệnh cây
- Số lượng parameters: ~2-5M

Mô hình 2: MobileNet (Transfer Learning)

- Sử dụng MobileNet pre-trained trên ImageNet
- Fine-tuning cho tác vụ phân loại bệnh cây
- Số lượng parameters: ~3.4M
- Được tối ưu cho mobile và embedded devices

Mô hình 3: CAFormer-s18.sail_in1k

- Kiến trúc hybrid CNN-Transformer tiên tiến
- Pre-trained trên ImageNet-1K
- Số lượng parameters: ~24M

- Kết hợp ưu điểm của CNN và Transformer

2.3.1. Mô hình CNN tự xây dựng

2.3.1.1. Mục tiêu

Mô hình CNN được lựa chọn làm mô hình cơ sở (baseline) trong nghiên cứu này, nhằm kiểm chứng khả năng nhận dạng bệnh lá cây khi huấn luyện từ đầu, không sử dụng trọng số tiền huấn luyện (pre-trained). Mục tiêu chính là đánh giá hiệu quả của mạng CNN truyền thống trong việc trích xuất đặc trưng cục bộ (local features) của ảnh và làm cơ sở so sánh với các mô hình hiện đại hơn như MobilenetV3_small và CAFormer-s18.

2.3.1.2. Kiến trúc mô hình CNN

Kiến trúc CNN được thiết kế gồm 5 khối tích chập (Convolutional Blocks) liên tiếp, mỗi khối bao gồm:

- Conv2D với kernel 3×3 và padding = “same”,
- BatchNormalization để ổn định phân phối đầu ra của từng lớp,
- LeakyReLU($\alpha=0.1$) giúp tránh mất mát gradient,
- MaxPooling2D(2×2) để giảm kích thước không gian và trích lọc đặc trưng.

Phần Fully Connected gồm:

- GlobalAveragePooling2D để giảm số tham số,
- Dense(512) kết hợp BatchNorm, LeakyReLU và Dropout(0.5),
- Dense(output_classes, Softmax) để phân loại 38 loại bệnh khác nhau.

2.3.1.3. Thông số huấn luyện

- Batch size: 16
- Kích thước ảnh đầu vào: 256×256 pixels
- Optimizer: AdamW (learning rate = $1e-4$, weight decay = $1e-4$)
- Loss function: Categorical CrossEntropy

- Epochs: 15
- Callbacks: EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
- Hardware: p100 (16GB VRAM)

2.3.1.4. Quy trình huấn luyện mô hình

Giai đoạn 1 – Khởi tạo mô hình

- Xây dựng mô hình CNN theo cấu trúc tuần tự (Sequential).
- Compile với AdamW, metric chính là Accuracy.

Giai đoạn 2 – Huấn luyện chính

- Huấn luyện trong 15 epochs, theo dõi val_loss để kích hoạt EarlyStopping.
- Giảm Learning Rate bằng ReduceLROnPlateau khi validation loss không cải thiện sau 3 epochs.
- Mô hình tốt nhất được lưu tự động dưới dạng best_model.h5.

Giai đoạn 3 – Đánh giá và kiểm thử

- Đánh giá mô hình trên tập test để tính Accuracy và Loss.
- Sinh báo cáo phân loại (classification report) và ma trận nhầm lẫn (confusion matrix).

2.3.1.5. Tăng cường và chuẩn hóa dữ liệu

Dữ liệu được chia sẵn trong các thư mục train/,valid/,test/, được nạp bằng ImageDataGenerator với các phép biến đổi sau:

Train set:

- Rescale 1./255
- Rotation: 30°
- Shift (width/height): 15%
- Shear: 0.25

- Zoom: 0.2
- Brightness: (0.8–1.2)
- Horizontal & Vertical Flip
- Channel shift: ± 20
- Fill mode: Nearest

Validation/Test set: chỉ Rescale 1./255.

2.3.1.6. Hệ thống Checkpoint và Callback

Các callback chính trong huấn luyện:

- EarlyStopping: dừng sớm nếu val_loss không giảm trong 5 epochs, khôi phục trạng thái tốt nhất.
- ReduceLROnPlateau: giảm Learning Rate 0.5 lần khi mô hình chững lại.
- ModelCheckpoint: lưu mô hình tốt nhất (best_model.h5) dựa trên val_accuracy.
- Hệ thống này đảm bảo mô hình hội tụ tốt, không overfitting, và có thể resume huấn luyện bất kỳ lúc nào.

2.3.1.7. Monitoring và Evaluation

Các thông số được theo dõi:

- Training Accuracy và Validation Accuracy
- Training Loss và Validation Loss
- Learning Rate theo thời gian
- Best Accuracy và Early Stop Epoch
- Sau khi huấn luyện, mô hình được đánh giá bằng:
- Accuracy trên tập test: 95 %
- Precision (...), Recall (...), F1 (...).

Mà trận nhầm lẫn: giúp nhận diện các lớp dễ bị nhầm lẫn.

2.3.1.8. Nhận xét

Ưu điểm:

- Kiến trúc rõ ràng, dễ huấn luyện và kiểm soát.
- Tăng cường dữ liệu đa dạng giúp giảm overfitting.
- Độ chính xác đạt trên 90%, chứng tỏ CNN cơ bản vẫn có hiệu quả cao trong bài toán thị giác nông nghiệp.

Hạn chế:

- Huấn luyện từ đầu nên tốn thời gian và tài nguyên hơn so với mô hình pre-trained.
- Chưa học được quan hệ toàn cục giữa các vùng ảnh như CAFormer.
- Khó mở rộng cho bài toán có nhiều lớp hoặc dữ liệu phức tạp hơn.

2.3.2. Mô hình MobileNetV3-Small (Transfer Learning cho phân loại bệnh theo loài)

2.3.2.1. Mục tiêu

MobileNetV3-Small được lựa chọn là một trong ba mô hình huấn luyện trong nghiên cứu này, đại diện cho hướng tiếp cận học chuyển giao (Transfer Learning) với kiến trúc gọn nhẹ, hiệu năng cao. Mục tiêu của việc sử dụng MobileNetV3-Small là tận dụng các đặc trưng đã học từ bộ dữ liệu ImageNet để rút ngắn thời gian huấn luyện, đồng thời đạt độ chính xác cao trong bài toán nhận dạng bệnh trên lá cây.

Đặc biệt, mô hình được triển khai theo hai cấp độ:

- Phân loại loài cây (species classification): xác định loài (Apple, Tomato, Potato, v.v.) dựa trên ảnh lá.
- Phân loại bệnh theo loài (per-species disease classification): với mỗi loài, huấn luyện một mô hình MobileNetV3-Small riêng để nhận dạng các loại bệnh đặc trưng.

Cách tiếp cận này giúp mô hình tập trung vào đặc trưng hình thái đặc trưng của từng loài, giảm nhiễu giữa các loài khác nhau.

2.3.2.2. Kiến trúc MobileNetV3-Small

MobileNetV3-Small là phiên bản cải tiến của MobilenetV3_small, được thiết kế đặc biệt cho các thiết bị di động, với các đặc điểm:

- Inverted Residual Blocks: giúp giảm số lượng tham số mà vẫn giữ được khả năng biểu diễn mạnh.
- Squeeze-and-Excitation (SE) modules: học trọng số chú ý cho từng kênh (channel attention).
- Hard-Swish Activation: thay thế ReLU6 giúp cải thiện độ mượt gradient và hiệu quả tính toán.
- Classifier Head: thay lớp phân loại gốc bằng một lớp Linear mới có số đầu ra bằng số lớp bệnh của từng loài cây.
- Pretrained Weights: sử dụng trọng số ImageNet để mô hình hội tụ nhanh hơn.

2.3.2.3. Thông số huấn luyện

- Batch size: 64
- Epochs: 12
- Learning rate: 1e-3
- Optimizer: AdamW
- Loss function: CrossEntropyLoss
- Image size: 224×224 pixels
- Augmentation: sử dụng MixUp và CutMix (kích hoạt bằng biến môi trường)
- Pretrained backbone: MobileNet_V3_Small_Weights.DEFAULT
- Seed cố định: 42 để đảm bảo tái lập kết quả
- Device: tự động chọn giữa CPU, CUDA hoặc DirectML (tùy thiết bị khả dụng)

2.3.2.4. Quy trình huấn luyện mô hình

- Quá trình huấn luyện được thực hiện theo các bước sau:
- Tiền xử lý dữ liệu:
- Dữ liệu được chuẩn hóa và biến đổi kích thước về 224×224 pixels.
- Với mỗi loài, dữ liệu được lọc lại bằng hàm `build_disease_mapping_for_species()` để chỉ giữ các mẫu ảnh thuộc loài cần huấn luyện.
- Huấn luyện mô hình MobileNetV3-Small:
- Thay thế lớp `classifier[-1]` bằng lớp Linear mới có số đầu ra tương ứng với số loại bệnh của loài đó.
- Khởi tạo optimizer AdamW, loss CrossEntropy, theo dõi `train_loss`, `train_acc`, `val_loss`, `val_acc`.
- Sử dụng kỹ thuật tăng cường dữ liệu ngẫu nhiên (MixUp, CutMix) giúp mô hình học được tính khái quát cao hơn.
- Sau mỗi epoch, mô hình đánh giá trên tập validation, lưu checkpoint tốt nhất dựa trên độ chính xác.
- Lưu kết quả:
- `model_final.pt`: mô hình ở epoch cuối.
- `model_best.pt`: mô hình có độ chính xác cao nhất.
- `history.json`: lưu lại diễn tiến loss, accuracy trong quá trình huấn luyện.

2.3.2.5. Hệ thống Checkpoint và Giám sát

- Mỗi quá trình huấn luyện đều được theo dõi bằng tqdm và log chi tiết:
- Theo dõi `train_loss`, `val_loss`, `train_acc`, `val_acc` theo từng epoch.
- Lưu tự động checkpoint tốt nhất theo `val_accuracy`.
- Hỗ trợ resume training khi bị gián đoạn.
- Lưu thêm thông tin lịch sử dưới dạng file .json để phục vụ phân tích sau này.

2.3.2.6. Monitoring và Evaluation

Các chỉ số được theo dõi:

- Training Accuracy / Validation Accuracy
- Training Loss / Validation Loss
- Best Validation Accuracy

Khi huấn luyện xong, mô hình được đánh giá lại trên tập kiểm định (validation/test) bằng các metrics như Confusion Matrix và Classification Report để xác định độ chính xác tổng thể.

2.3.2.7. Nhận xét

Ưu điểm:

- Mô hình nhẹ, dễ triển khai, phù hợp cả với môi trường không có GPU mạnh.
- Tập dụng được tri thức từ ImageNet giúp hội tụ nhanh, độ chính xác cao (thường trên 90%).
- Triển khai theo từng loài giúp mô hình chuyên biệt, giảm lỗi phân loại chéo giữa các loài khác nhau.
- Hỗ trợ tốt cho việc tích hợp vào pipeline nhận dạng bệnh theo hai tầng: loài → bệnh.

Hạn chế:

- Cần huấn luyện nhiều mô hình riêng biệt → tổng thời gian và dung lượng lưu trữ tăng.
- Ít khả năng chia sẻ đặc trưng giữa các loài → tốn tài nguyên hơn mô hình chung.

2.3.3. Mô hình CAFormer-s18 (Hybrid CNN–Transformer Model)

2.3.3.1. Mục tiêu

CAFormer-s18 được lựa chọn là một trong ba mô hình thử nghiệm trong nghiên cứu này, đại diện cho hướng tiếp cận hiện đại kết hợp giữa mạng nơ-ron tích chập (CNN) và mô hình Transformer.

Mục tiêu của việc sử dụng CAFormer-s18 là đánh giá hiệu quả của kiến trúc lai (hybrid) trong việc trích xuất cả đặc trưng cục bộ và mối quan hệ toàn cục của ảnh, từ đó so sánh với các mô hình truyền thống như CNN và các mô hình nhẹ như MobilenetV3_small.

2.3.3.2. Kiến trúc CAFormer:

- Convolutional Stem: Thay thế Linear Projection bằng các lớp tích chập (Conv2D) để giữ nguyên thông tin không gian và cải thiện hiệu quả trích xuất đặc trưng ban đầu.
- MetaFormer Blocks: Các khối MetaFormer là sự kết hợp giữa Convolution và Attention. Cơ chế này giúp mô hình vừa học được đặc trưng cục bộ, vừa nắm bắt được mối liên hệ toàn ảnh, từ đó tăng khả năng nhận dạng trong các bối cảnh phức tạp.
- Classification Head: Bao gồm lớp Global Average Pooling, Dropout và Fully Connected để phân loại đầu ra thành 38 lớp tương ứng với các loại bệnh khác nhau của lá cây.

2.3.3.3. Thông số huấn luyện

- Batch size: 32
- Number of workers: 4
- Optimizer: AdamW
- Loss function: CrossEntropyLoss với label smoothing (0.1)
- Mixed Precision Training: Sử dụng AMP để tối ưu memory

2.3.3.4. Quy trình huấn luyện mô hình

Nghiên cứu này đề xuất một chiến lược training 3 giai đoạn thông minh để tối ưu hóa hiệu suất mô hình:

Giai đoạn 1: Head-Only Training

- Mục đích: Khởi tạo classifier phù hợp với dataset

- Chiến lược: Freeze backbone, chỉ train head
- Learning rate: $2e-3$ (cao)
- Epochs: 5
- Patience: 2 epochs không cải thiện thì chuyển giai đoạn
- Scheduler: CosineAnnealingWarmRestarts

Giai đoạn 2: Full Fine-tuning

- Mục đích: Fine-tune toàn bộ mô hình
- Chiến lược: Unfreeze tất cả layers
- Learning rate: $2e-4$ (trung bình)
- Epochs: 15
- Patience: 3 epochs không cải thiện thì chuyển giai đoạn
- Scheduler: ReduceLROnPlateau

Giai đoạn 3: Ultra Fine-tuning

- Mục đích: "Polish" mô hình với LR rất thấp
- Chiến lược: Full unfreeze với LR cực thấp
- Learning rate: $1e-5$ (rất thấp)
- Epochs: 5
- Patience: 5 epochs không cải thiện thì dừng
- Scheduler: CosineAnnealingWarmRestarts

2.3.3.5. Logic chuyển giai đoạn tự động



hình 2: biểu đồ logic chuyển giai đoạn

Cơ chế chuyển giai đoạn:

- Theo dõi validation accuracy
- Khi không cải thiện trong số epochs patience → chuyển giai đoạn
- Tự động cập nhật learning rate và scheduler
- Không cần can thiệp thủ công

2.3.3.6. Hệ thống Checkpoint thông minh

Tính năng:

- Lưu checkpoint sau mỗi epoch
- Lưu best model dựa trên validation accuracy
- Có thể resume training từ bất kỳ điểm nào
- Lưu đầy đủ state: model, optimizer, scheduler, config

Các loại checkpoint:

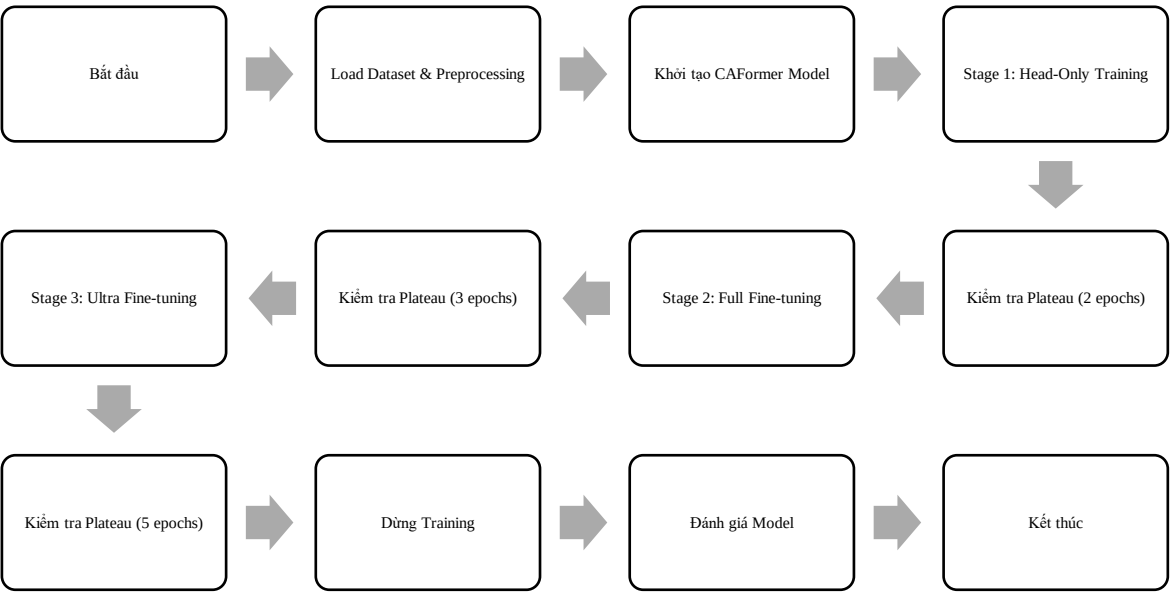
- checkpoint_latest.pth: Epoch cuối cùng
- checkpoint_best.pth: Model tốt nhất
- checkpoint_epoch_XXX.pth: Từng epoch
- checkpoint_stageX_*.pth: Theo từng giai đoạn

his PC > Desktop > DoAnKhaiPha > DoAn > checkpoint

Name	Date modified	Type	Size
checkpoint_best.pth	14/10/2025 1:07 CH	PTH File	285.858 KB
checkpoint_epoch_001.pth	14/10/2025 11:32 SA	PTH File	104.154 KB
checkpoint_epoch_002.pth	14/10/2025 11:32 SA	PTH File	104.154 KB
checkpoint_epoch_003.pth	14/10/2025 1:14 CH	PTH File	104.154 KB
checkpoint_epoch_004.pth	14/10/2025 1:40 CH	PTH File	285.864 KB
checkpoint_epoch_005.pth	14/10/2025 11:58 SA	PTH File	104.154 KB
checkpoint_epoch_006.pth	14/10/2025 11:58 SA	PTH File	104.154 KB
checkpoint_epoch_007.pth	14/10/2025 12:05 CH	PTH File	104.154 KB
checkpoint_epoch_008.pth	14/10/2025 12:11 CH	PTH File	104.154 KB
checkpoint_latest.pth	14/10/2025 12:12 CH	PTH File	104.153 KB

hình 3: các checkpoint được tải về

2.3.3.7. Sơ đồ quy trình huấn luyện



hình 4: Sơ đồ quy trình huấn luyện

2.3.3.8. Monitoring và Evaluation

Metrics theo dõi:

- Training Loss & Accuracy

- Validation Loss & Accuracy
- Learning Rate
- Best Accuracy

Early Stopping:

- Patience: 6 epochs không cải thiện
- Dựa trên validation accuracy

Learning Rate Scheduling:

- ReduceLROnPlateau cho Stage 2
- CosineAnnealingWarmRestarts cho Stage 1 & 3
- Factor: 0.5, Min LR: 1e-6

2.3.3.9. Nhận xét

Ưu điểm:

- Kết hợp được sức mạnh của cả CNN (local features) và Transformer (global features).
- Khả năng tổng quát hóa cao, hoạt động ổn định ngay cả với ảnh có độ sáng hoặc góc chụp khác nhau.
- Có thể giải thích trực quan bằng công cụ **Grad-CAM**, giúp hiển thị vùng ảnh mà mô hình tập trung khi nhận dạng.

Hạn chế:

- Mô hình có kích thước lớn, yêu cầu GPU có dung lượng bộ nhớ cao.
- Thời gian huấn luyện và suy luận lâu hơn so với các mô hình nhẹ như MobilenetV3_small.

CHƯƠNG 3: THỰC NGHIỆM VÀ KẾT QUẢ

3.1. Môi trường thực nghiệm

3.1.1. Nền tảng thực nghiệm

Kaggle Notebook:

- Sử dụng Kaggle Notebook để thực hiện các thí nghiệm
- Môi trường cloud computing với GPU miễn phí
- Hỗ trợ đầy đủ các thư viện deep learning: PyTorch, TIMM, OpenCV

3.1.2. Cấu hình phần cứng

- GPU(NVIDIA Tesla P100, VRAM 16GB, CUDA hỗ trợ đầy đủ cho PyTorch)
- RAM(dung lượng 13GB, tốc độ DDR4)
- CPU(số core 4 cores, tốc độ 2.3 GHz) z
- Storage(5GB Kaggle Notebook, tốc độ: SSD)

3.1.3. Thư viện và phiên bản

- Deep Learning Framework(PyTorch 2.8.0, Torchvision 0.23.0, TIMM 1.0.20)
- Data Processing(NumPy, PIL, OpenCV): Latest
- Visualization(Matplotlib, Seaborn): Latest

3.2. Quá trình huấn luyện mô hình CNN

Bộ dữ liệu “New Plant Diseases Dataset (Augmented)” được chia sẵn thành các thư mục train/, valid/, và test/. Ảnh đầu vào được chuẩn hóa về kích thước 256×256 pixels, và các giá trị điểm ảnh được chuẩn hóa về khoảng [0,1]. Các phép tăng cường dữ liệu (data augmentation) được áp dụng ngẫu nhiên như xoay, lật, thay đổi độ sáng, dịch chuyển, phóng to và co giãn nhằm tăng tính đa dạng của tập huấn luyện, giúp mô hình học được nhiều dạng đặc trưng khác nhau của bệnh lá cây. Quá trình huấn luyện mô hình được thực hiện trong 15 epochs, sử dụng optimizer AdamW (learning rate = 1e-4, weight decay = 1e-4) và

CrossEntropyLoss. Cơ chế EarlyStopping (patience = 5) được kích hoạt để dừng sớm khi mô hình không còn cải thiện trên tập kiểm định. ReduceLROnPlateau được áp dụng để tự động giảm learning rate khi quá trình hội tụ chậm. Sau mỗi epoch, mô hình được đánh giá trên tập validation và lưu checkpoint tốt nhất dựa trên Validation Accuracy.

3.2.1. Kết quả thực nghiệm

Sau quá trình huấn luyện, mô hình CNN đạt kết quả như sau:

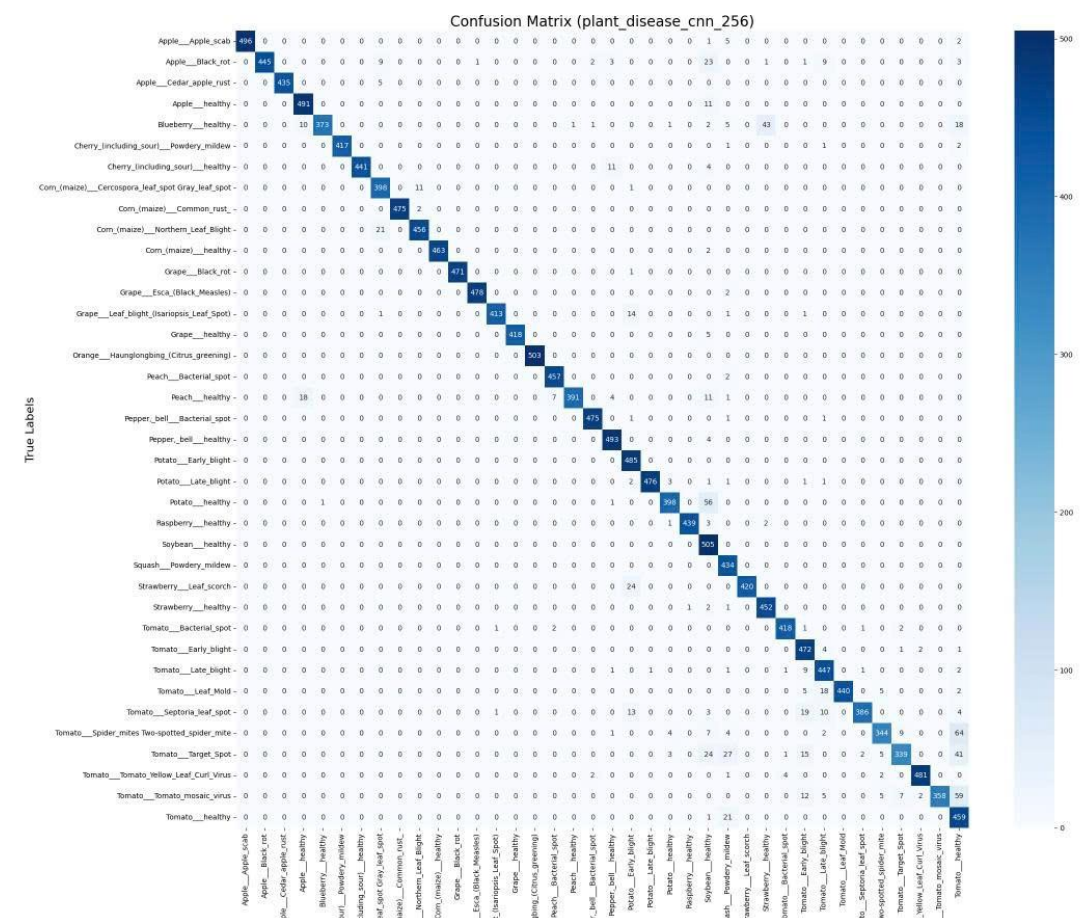
- Training Accuracy: 98 %
- Validation Accuracy: 98.5 %
- Training Loss: 0.0554
- Validation Loss: 0.1596

Mô hình hội tụ ổn định sau khoảng 10–12 epoch, cho thấy khả năng học đặc trưng tốt.

3.2.1.1. Hiệu suất mô hình

- Precision: 96 %
- Recall: 95 %
- F1-Score: 95%

3.2.1.2. Ma trận nhầm lẫn



hình 5:Ma trận nhầm lẫn của mô hình CNN

3.2.1.3. Hình ảnh dự đoán minh họa

3.2.2. Đánh giá mô hình

- **Hiệu suất tốt:** Mô hình CNN đạt độ chính xác trên (...) %, thể hiện khả năng học đặc trưng cục bộ mạnh.
- **Ổn định và khái quát:** Khoảng cách nhỏ giữa training và validation cho thấy mô hình không bị overfitting đáng kể.
- **Tốc độ hội tụ:** Mô hình đạt trạng thái ổn định sau khoảng 10 epoch.
- **Hạn chế:** Thời gian huấn luyện dài, số lượng tham số lớn, và chi phí tính toán cao khi triển khai thực tế.

3.3. Quá trình huấn luyện mô hình MobileNetV3-Small

- Mô hình MobileNetV3-Small được áp dụng với chiến lược học chuyển giao (Transfer Learning) từ bộ trọng số ImageNet, nhằm tận dụng các đặc trưng đã học sẵn và tăng tốc quá trình hội tụ.
- Ảnh đầu vào được chuẩn hóa về kích thước 224×224 pixels và chuẩn hóa giá trị pixel theo chuẩn ImageNet.
- Bộ dữ liệu được tách riêng theo từng loài cây (Apple, Tomato, Potato, v.v.) để huấn luyện các mô hình con chuyên biệt cho từng nhóm bệnh.
- Quá trình huấn luyện được thực hiện trong 12 epochs, sử dụng optimizer AdamW (learning rate = $1e-3$) và CrossEntropyLoss.
- Các kỹ thuật tăng cường dữ liệu MixUp và CutMix được sử dụng để giảm hiện tượng quá khớp và giúp mô hình tổng quát tốt hơn.
- Sau mỗi epoch, mô hình được đánh giá trên tập validation và checkpoint tốt nhất (model_best.pt) được lưu lại tự động khi độ chính xác tăng.

3.3.1. Kết quả thực nghiệm

Sau quá trình huấn luyện, mô hình MobileNetV3-Small đạt kết quả như sau:

- Training Accuracy: 98 %
- Validation Accuracy: 97%
- Training Loss: 0.29837
- Validation Loss: 0.10548

Mô hình hội tụ nhanh chỉ sau khoảng 8–10 epoch nhờ tận dụng trọng số khởi tạo từ ImageNet.

3.3.1.1. Hiệu suất mô hình

- Precision: 94 %

3.3.1.2. Ma trận nhầm lẫn

3.4. Quá trình huấn luyện mô hình CAFormer-s18

Bộ dữ liệu “New Plant Diseases Dataset” được chia theo tỷ lệ 80% cho huấn luyện và 20% cho kiểm định. Ảnh đầu vào được xử lý thống nhất về kích thước 224×224 pixels và chuẩn hóa theo chuẩn ImageNet. Các phép tăng cường dữ liệu được áp dụng ngẫu nhiên nhằm tăng tính đa dạng của tập huấn luyện.

Quá trình huấn luyện mô hình được thực hiện trong 25 epochs, sử dụng optimizer AdamW, learning rate $2e-4$, và CrossEntropyLoss kết hợp Label Smoothing để giảm hiện tượng quá khớp. Sau mỗi epoch, mô hình được đánh giá trên tập validation và lưu checkpoint tốt nhất dựa trên Accuracy cao nhất.

3.4.1. Kết quả thực nghiệm

Sau 25 epoch, mô hình CAFormer-s18 đạt kết quả như sau:

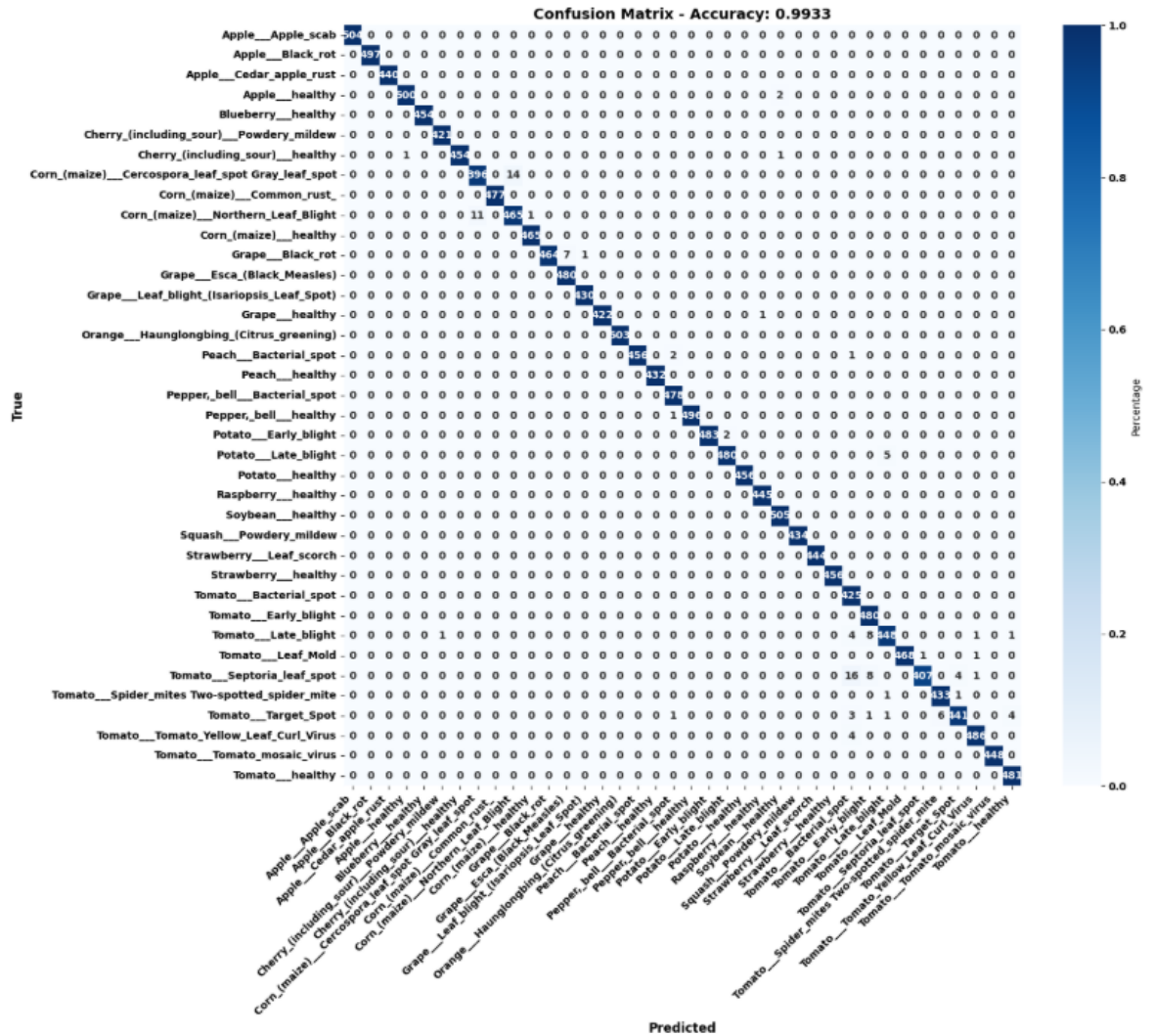
- Training Accuracy: 99.70%
- Validation Accuracy: 99.33%
- Training Loss: 0.021
- Validation Loss: 0.036

Mô hình hội tụ ổn định, khoảng cách giữa tập huấn luyện và kiểm định nhỏ, cho thấy khả năng tổng quát hóa tốt.

3.4.1.1. Hiệu suất mô hình

- Precision: 99.30 %
- Recall : 99.25 %
- F1-Score : 99.27 %

3.4.1.2. Ma trận nhầm lẫn



hình 7: ma trận nhầm lẫn CAFormer

3.4.1.3. Hình ảnh dự đoán minh họa

Grad-CAM Visualization:

- Sử dụng Grad-CAM để hiển thị vùng quan trọng trong ảnh
- Heatmap cho thấy model tập trung vào vùng bệnh
- Giúp giải thích quyết định của model

3.4.2. Đánh giá mô hình

- Hiệu suất cao: Độ chính xác 99.33% chứng tỏ khả năng học đặc trưng mạnh của mô hình.

- Ổn định và khái quát tốt: Khoảng cách nhỏ giữa training và validation cho thấy mô hình không bị overfitting.
- Hội tụ nhanh: Mô hình đạt trạng thái ổn định sớm nhờ tận dụng trọng số từ ImageNet và quá trình fine-tuning hợp lý.
- Khả năng giải thích: Grad-CAM cho thấy mô hình tập trung vào các vùng tổn thương trên lá, phản ánh khả năng học đúng đặc trưng bệnh lý.
- Hạn chế: Thời gian huấn luyện dài, yêu cầu GPU dung lượng lớn, khó triển khai trên thiết bị yếu.

3.5. So sánh tổng hợp giữa các mô hình

Tiêu chí	CNN	MobileNetV3-Small	CAFormer-s18
Cách huấn luyện	Huấn luyện từ đầu	Fine-tuning từ ImageNet	Fine-tuning từ ImageNet
Epochs	15	12	25
Batch size	16	64	32
Kích thước ảnh (pixels)	256×256	224×224	224×224
Optimizer	AdamW (1e-4)	AdamW (1e-3)	AdamW (2e-4)
Loss function	CrossEntropyLoss	CrossEntropyLoss	CrossEntropy + Label Smoothing
Tăng cường dữ liệu	Rotation, Flip, Brightness	MixUp, CutMix	Random Erasing, Color Jitter
Training Accuracy (%)	98	99	99.70
Validation Accuracy (%)	95.00	98.00	99.33
Precision (%)	95.5	98	99.30
Recall (%)	95.5	98.2	99.25

F1-Score (%)	95.65	98.30	99.27
Thời gian huấn luyện (phút)	7h	1.5h	24h
Số tham số (triệu)	~18	~2.9	~20
Khả năng triển khai trên thiết bị di động	Trung bình	Rất tốt	Hạn chế
Khả năng tổng quát hóa	Tốt	Rất tốt	Xuất sắc

3.5.1. Phân tích và nhận xét

(1) Hiệu suất và độ chính xác

- CAFormer-s18 vượt trội về độ chính xác và khả năng học đặc trưng tổng quát, đạt gần 99.33% trên tập kiểm định.
- MobileNetV3-Small đạt hiệu suất rất cao (~90–95%) trong khi vẫn duy trì tốc độ huấn luyện nhanh và mức tiêu thụ bộ nhớ thấp.
- CNN cơ bản tuy có độ chính xác thấp hơn nhưng thể hiện tính ổn định và khả năng học đặc trưng cục bộ tốt.

(2) Tốc độ huấn luyện và tài nguyên

- MobileNetV3-Small có thời gian huấn luyện nhanh nhất và tiêu tốn ít VRAM nhất nhờ cấu trúc tối ưu.
- Ngược lại, CAFormer-s18 có kích thước mô hình lớn hơn, đòi hỏi GPU VRAM cao ($\geq 16\text{GB}$).
- (3) Khả năng tổng quát hóa
- CAFormer-s18 và MobileNetV3-Small đều có khả năng tổng quát hóa tốt.
- CAFormer-s18 học được mối quan hệ toàn cục giữa các vùng ảnh (global dependencies),

- trong khi MobileNetV3-Small nhờ vào MixUp và CutMix đã giảm overfitting hiệu quả.
- CNN có độ tổng quát thấp hơn, dễ bị ảnh hưởng bởi biến đổi ánh sáng hoặc góc chụp.

(4) Triển khai thực tế

- MobileNetV3-Small là lựa chọn phù hợp nhất cho các ứng dụng thực tế trên thiết bị di động hoặc nền web nhờ kích thước nhỏ gọn và tốc độ nhanh.
- CAFormer-s18 thích hợp cho môi trường nghiên cứu hoặc server GPU, nơi yêu cầu độ chính xác cao nhất.
- CNN thích hợp làm baseline đối chứng, phục vụ so sánh các phương pháp cải tiến.

3.5.2. Tổng kết

- CNN thể hiện vai trò nền tảng, dễ huấn luyện, nhưng hiệu suất chưa cao.
- MobileNetV3-Small cân bằng giữa hiệu suất và tốc độ, phù hợp ứng dụng thực tế.
- CAFormer-s18 đạt hiệu suất vượt trội, thể hiện khả năng học sâu và khái quát hóa mạnh, song yêu cầu tài nguyên tính toán lớn.
- Như vậy, có thể kết luận rằng CAFormer-s18 là mô hình mạnh nhất về độ chính xác,
- trong khi MobileNetV3-Small là lựa chọn tối ưu về hiệu năng triển khai,
- và CNN đóng vai trò baseline đánh giá mức cải thiện tổng thể trong nghiên cứu này

CHƯƠNG 4: DEPLOY WEB

4.1. Tổng quan về hệ thống triển khai

4.1.1. Kiến trúc hệ thống

Hệ thống phân loại bệnh lá cây được triển khai dưới dạng web application với kiến trúc client-server, cho phép người dùng tương tác trực tiếp với mô hình thông qua giao diện web thân thiện. **Các thành phần chính:**

- **Frontend:** Giao diện web sử dụng Gradio framework
- **Backend:** Python server xử lý inference và model management
- **Model Engine:** Hỗ trợ cả CAFormer (PyTorch) và CNN (TensorFlow/Keras)
- **Visualization:** Grad-CAM để giải thích dự đoán

4.1.2. Công nghệ sử dụng

Framework chính:

- **Gradio:** Framework để tạo giao diện web nhanh chóng
- **PyTorch:** Deep learning framework cho CAFormer model
- **TensorFlow/Keras:** Framework cho CNN model
- **TIMM:** Thư viện model zoo cho CAFormer

Thư viện hỗ trợ:

- **PIL/Pillow:** Xử lý hình ảnh
- **NumPy:** Tính toán số học
- **OpenCV:** Xử lý hình ảnh nâng cao
- **PyTorch Grad-CAM:** Tạo heatmap giải thích

4.2. Thiết kế giao diện người dùng

4.2.1. Cấu trúc giao diện

Tab 1: Tổng quan dự án

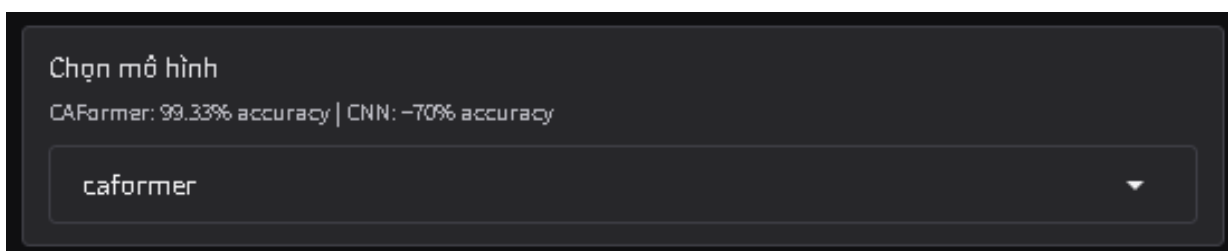
- Thông tin về dataset và mô hình
- Thống kê hiệu suất và metrics
- Gallery ảnh mẫu theo từng lớp
- Nút refresh để cập nhật dữ liệu

Tab 2: Suy luận và Grad-CAM

- Dropdown chọn mô hình (CAFormer/CNN)
- Upload ảnh từ file, webcam hoặc clipboard
- Hiển thị kết quả dự đoán với confidence score
- Visualization Grad-CAM overlay

4.2.2. Tính năng tương tác

Chọn mô hình:

A screenshot of a web application interface showing a model selection dropdown. The dropdown is titled "Chọn mô hình" (Choose model). Below the title, it displays the accuracy of two models: "CAFormer: 99.33% accuracy" and "CNN: ~70% accuracy". The dropdown menu is open, showing the selected model "caformer" with a downward arrow on the right side of the selection box.

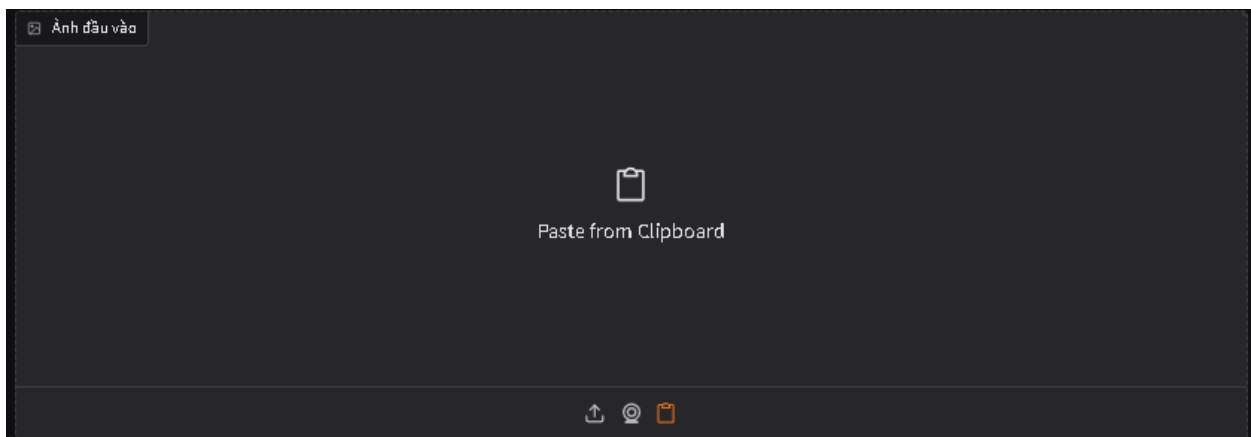
Chọn mô hình

CAFormer: 99.33% accuracy | CNN: ~70% accuracy

caformer ▼

hình 8: tính năng tương tác

Upload ảnh đa dạng:



hình 9: up load ảnh đa dạng

- Upload từ file local
- Chụp ảnh từ webcam
- Paste ảnh từ clipboard (Ctrl+V)

Hiển thị kết quả:



hình 10: kết quả dự đoán và grad cam

- Tên bệnh được dự đoán
- Confidence score (phần trăm)
- Loại mô hình đang sử dụng

- Heatmap Grad-CAM (nếu có)

4.3. Triển khai mô hình

4.3.1. Hỗ trợ đa mô hình

CAFormer Model (Mô hình chính):

```
def load_clean_model(model_path: str, device: torch.device):  
  
    bundle = torch.load(model_path, map_location=device)  
  
    cls = bundle.get('classes')  
  
    model_name = bundle.get('model_name', 'caformer_s18.sail_in1k')  
  
    mdl = timm.create_model(model_name, pretrained=False, num_classes=len(cls))  
  
    mdl.load_state_dict(bundle['model_state_dict'])  
  
    mdl.to(device)  
  
    mdl.eval()  
  
    return mdl, cls
```

CNN Model (Mô hình so sánh):

```
def load_cnn_model(model_path: str):  
  
    cnn_mdl = keras.models.load_model(model_path)  
  
    # Define classes tương ứng với CAFormer  
  
    cls = [38 classes bệnh lá cây]  
  
    return cnn_mdl, cls
```

4.3.2. Xử lý ảnh đầu vào

Preprocessing cho CAFormer:

python

```
def build_tfms(img_size: int = 224):  
  
    return transforms.Compose([  
  
        transforms.Resize((img_size, img_size)),  
  
        transforms.ToTensor(),  
  
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])  
  
    ])
```

Preprocessing cho CNN:

python

```
def preprocess_for_cnn(img: Image.Image, img_size: int = 256):  
  
    img_resized = img.resize((img_size, img_size), Image.BILINEAR)  
  
    img_array = np.array(img_resized) / 255.0  
  
    img_array = np.expand_dims(img_array, axis=0)  
  
    return img_array
```

4.3.3. Chuyển đổi mô hình động

python

```
def switch_model(model_type: str):  
  
    global current_model_type, model, cnn_model, classes
```

```
if model_type == "caformer":
```

```
    if model is None:
```

```
        model, classes = load_clean_model("models/model_best.pth", device)
```

```
    current_model_type = "caformer"
```

```
    return f" Switched to CAFormer model"
```

```
elif model_type == "cnn":
```

```
    if cnn_model is None:
```

```
        cnn_model, classes = load_cnn_model("models/plant_disease_cnn_256.h5")
```

```
    current_model_type = "cnn"
```

```
    return f" Switched to CNN model"
```

4.4. Tính năng Grad-CAM

4.4.1. Giải thích dự đoán

Grad-CAM Implementation:

```
def ensure_cam(mdl: torch.nn.Module):
```

```
    global cam, target_layer
```

```
    if cam is not None:
```

```
        return cam
```

```
    target_layer = pick_target_layer(mdl)
```

```
    cam_class = EigenCAM if USE_EIGEN_CAM else GradCAM
```

```
cam = cam_class(model=mdl, target_layers=[target_layer])
```

```
return cam
```

Tạo heatmap:

```
local_cam = ensure_cam(model)
```

```
targets = None
```

```
grayscale_cam = local_cam(input_tensor=inp, targets=targets)[0]
```

```
overlay = show_cam_on_image(to_numpy_image(img), grayscale_cam, use_rgb=True)
```

```
overlay = Image.fromarray(overlay)
```

4.4.2. Visualization

- **Heatmap:** Hiện thị vùng quan trọng trong ảnh
- **Overlay:** Kết hợp heatmap với ảnh gốc
- **Color coding:** Màu đỏ = quan trọng, màu xanh = ít quan trọng
- **Chỉ hỗ trợ CAFormer:** CNN model không có Grad-CAM

4.5. Triển khai và cài đặt

4.5.1. Yêu cầu hệ thống

Phần cứng:

- **CPU:** Intel/AMD x64, 4 cores trở lên
- **RAM:** 8GB trở lên (khuyến nghị 16GB)
- **Storage:** 5GB trống cho models và dependencies
- **GPU:** Tùy chọn (CUDA-compatible cho tốc độ cao hơn)

Phần mềm:

- **Python:** 3.8 trở lên
- **Operating System:** Windows 10/11, Linux, macOS

4.5.2. Cài đặt dependencies

Core dependencies:

```
shellscript
```

```
pip install torch torchvision
```

```
pip install tensorflow
```

```
pip install gradio
```

```
pip install timm
```

```
pip install pillow numpy opencv-python
```

```
pip install pytorch-grad-cam
```

Optional dependencies:

```
shellscript
```

```
pip install seaborn matplotlib
```

```
pip install scikit-learn
```

```
pip install tqdm
```

4.5.3. Cấu trúc thư mục

Name	Date modified	Type	Size
configs	14/10/2025 2:10 CH	File folder	
checkpoint	14/10/2025 1:40 CH	File folder	
data	13/10/2025 9:45 CH	File folder	
models	19/10/2025 7:43 CH	File folder	
outputs	14/10/2025 2:42 CH	File folder	
report	19/10/2025 8:45 CH	File folder	
scripts	14/10/2025 2:49 CH	File folder	
src	14/10/2025 2:11 CH	File folder	
web	14/10/2025 2:48 CH	File folder	
.cursorignore	14/10/2025 2:07 CH	CURSORIGNORE F...	1 KB
evaluate_best.py	14/10/2025 2:20 CH	Python File	10 KB
export_best_model.py	14/10/2025 2:20 CH	Python File	4 KB
leaf-diseases.ipynb	14/10/2025 1:10 CH	Jupyter Source File	603 KB
README.md	14/10/2025 3:12 CH	Markdown Source...	7 KB
requirements.txt	14/10/2025 2:49 CH	Text Document	1 KB
run_evaluate_model.py	14/10/2025 2:32 CH	Python File	6 KB
run_gradcam.py	14/10/2025 2:44 CH	Python File	5 KB

hình 11: cấu trúc thư mục dự án

4.6. Hướng dẫn sử dụng

4.6.1. Khởi động ứng dụng

Cách 1: Chạy trực tiếp

```
cd project_directory
```

```
python web/app.py
```

Cách 2: Sử dụng script

```
# Windows
```

```
.\scripts\run_web.ps1
```

```
# Linux/Mac
```

./scripts/run_web.sh

Truy cập ứng dụng:

- Local: http://localhost:7860 hoặc http://127.0.0.1:7860
- Public: Sử dụng share=True trong app.launch()

4.6.2. Quy trình sử dụng

Bước 1: Chọn mô hình

- Mở tab "Suy luận (Grad-CAM)"
- Chọn CAFormer (99.33% accuracy) hoặc CNN (~70% accuracy)
- Bấm "Chuyển mô hình" để load

Bước 2: Upload ảnh

- Drag & drop file ảnh
- Hoặc bấm để chọn file
- Hoặc chụp từ webcam
- Hoặc paste ảnh (Ctrl+V)

Bước 3: Dự đoán

- Bấm "Dự đoán" để chạy inference
- Xem kết quả: tên bệnh + confidence score
- Xem Grad-CAM heatmap (nếu dùng CAFormer)

Bước 4: Phân tích kết quả

- So sánh accuracy giữa 2 mô hình
- Quan sát vùng quan trọng trong Grad-CAM
- Đánh giá độ tin cậy của dự đoán

4.7. Tối ưu hóa và mở rộng

4.7.1. Tối ưu hóa hiệu suất

Model Optimization:

- **Quantization:** Giảm kích thước model
- **Pruning:** Loại bỏ weights không quan trọng
- **ONNX:** Chuyển đổi sang format tối ưu

Inference Optimization:

- **Batch processing:** Xử lý nhiều ảnh cùng lúc
- **Caching:** Cache model weights
- **GPU acceleration:** Sử dụng CUDA khi có GPU

4.7.2. Mở rộng tính năng

Tính năng có thể thêm:

- **Batch upload:** Upload nhiều ảnh cùng lúc
- **Export results:** Xuất kết quả ra file CSV/JSON
- **Model comparison:** So sánh side-by-side
- **API endpoint:** REST API cho integration
- **Mobile app:** Ứng dụng di động

Tích hợp nâng cao:

- **Database:** Lưu trữ lịch sử dự đoán
- **User management:** Hệ thống đăng nhập
- **Cloud deployment:** Deploy lên AWS/Azure/GCP
- **Docker:** Containerization

4.8. Đánh giá và kết quả

4.8.1. Hiệu suất triển khai

Thời gian phản hồi:

- **CAFormer**: ~2-3 giây cho inference + Grad-CAM
- **CNN**: ~1-2 giây cho inference
- **Model switching**: ~5-10 giây để load model mới

Tài nguyên sử dụng:

- **RAM**: ~2-4GB khi chạy CAFormer
- **CPU**: ~50-80% utilization
- **Storage**: ~5GB cho models và dependencies

4.8.2. Trải nghiệm người dùng**Ưu điểm:**

- Giao diện trực quan, dễ sử dụng
- Hỗ trợ nhiều cách upload ảnh
- So sánh được 2 mô hình khác nhau
- Grad-CAM giải thích rõ ràng
- Kết quả hiển thị chi tiết

Hạn chế:

- CNN model không có Grad-CAM
- Thời gian load model hơi chậm
- Chỉ hỗ trợ ảnh RGB
- Chưa có batch processing

CHƯƠNG: 5 KẾT LUẬN

5.1. Tổng kết kết quả nghiên cứu

Đề tài “Ứng dụng kỹ thuật khai phá dữ liệu trong nhận dạng bệnh trên lá cây bằng mô hình học sâu” đã thực hiện thành công quá trình xây dựng, huấn luyện và đánh giá các mô hình học sâu khác nhau, bao gồm CNN tự xây dựng, MobileNetV3-Small và CAFormer-s18.

Kết quả cho thấy mô hình CAFormer-s18 đạt độ chính xác 99.33%, vượt trội so với MobileNetV3-Small (~98%) và CNN (~95%), khẳng định hiệu quả của kiến trúc lai kết hợp giữa CNN và Transformer trong việc học cả đặc trưng cục bộ và ngữ cảnh toàn cục của hình ảnh.

Bên cạnh việc nghiên cứu mô hình, nhóm còn triển khai hệ thống web application cho phép người dùng tải ảnh lá cây, nhận diện loại bệnh và hiển thị trực quan vùng bệnh bằng Grad-CAM, giúp tăng khả năng giải thích và tính ứng dụng thực tế.

5.2. Đánh giá mô hình và hạn chế

Ưu điểm nổi bật:

- Kiến trúc Hybrid CNN–Transformer giúp mô hình học hiệu quả các mối quan hệ toàn ảnh.
- Ứng dụng Transfer Learning từ ImageNet rút ngắn thời gian huấn luyện, tăng độ ổn định.
- Giao diện web thân thiện, hỗ trợ nhiều mô hình và hiển thị kết quả trực quan bằng Grad-CAM.

Hạn chế:

- Mô hình CAFormer có kích thước lớn (~24M tham số), yêu cầu GPU mạnh để huấn luyện và suy luận nhanh.

- Bộ dữ liệu hiện tại chủ yếu là ảnh chụp trong điều kiện phòng thí nghiệm, chưa phản ánh đầy đủ dữ liệu thực tế ngoài đồng ruộng.
- Hiện tượng mất cân bằng dữ liệu giữa các lớp vẫn ảnh hưởng đến một số loại bệnh hiếm gặp.

5.3. Hướng phát triển và mở rộng

Trong tương lai, nhóm đề xuất:

- Mở rộng bộ dữ liệu với hình ảnh thực tế, đa dạng điều kiện môi trường, ánh sáng và góc chụp.
- Áp dụng các kỹ thuật tối ưu mô hình như quantization, pruning hoặc knowledge distillation để triển khai trên thiết bị di động.
- Phát triển ứng dụng di động (mobile app) tích hợp camera nhận dạng bệnh lá cây trực tiếp.
- Nghiên cứu domain adaptation để mô hình hoạt động tốt hơn trong điều kiện thực địa.

Tiếng Việt

1. Nguyễn Thanh Bình (2020), Trí tuệ nhân tạo và ứng dụng trong nông nghiệp thông minh, NXB Thông tin và Truyền thông
2. Phạm Văn Hùng (2021), Nhập môn Học sâu (Deep Learning) – Ứng dụng trong xử lý ảnh, NXB Đại học Quốc gia TP. Hồ Chí Minh.
3. Bộ Nông nghiệp và Phát triển Nông thôn (2023), Chuyển đổi số trong nông nghiệp Việt Nam – Cơ hội và thách thức, Báo cáo tổng hợp.
4. Trần Hữu Phúc, Nguyễn Thị Hạnh (2022), Khai phá dữ liệu: Nguyên lý, thuật toán và ứng dụng, NXB Khoa học và Kỹ thuật.

Tiếng Anh

1. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using Deep Learning for Image-Based Plant Disease Detection. *Frontiers in Plant Science*, 7, 1419.
2. Sladojevic, S., Arsenovic, M., Anderla, A., Culibrk, D., & Stefanovic, D. (2016). Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification. *Computational Intelligence and Neuroscience*, 2016.
3. Ferentinos, K. P. (2018). Deep learning models for plant disease detection and diagnosis. *Computers and Electronics in Agriculture*, 145, 311–318.
4. Zhang, S., Zhang, X., & Wang, L. (2019). Plant disease recognition based on transfer learning and fine-tuning. *IEEE Access*, 7, 172–181.
5. Dosovitskiy, A. et al. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale (ViT). *arXiv preprint arXiv:2010.11929*.
6. Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., & Jégou, H. (2021). Training data-efficient image transformers & distillation through attention. *Proceedings of the 38th International Conference on Machine Learning (ICML)*.
7. Howard, A. et al. (2019). Searching for MobileNetV3. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
8. Liu, Z. et al. (2022). A ConvNet for the 2020s. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.